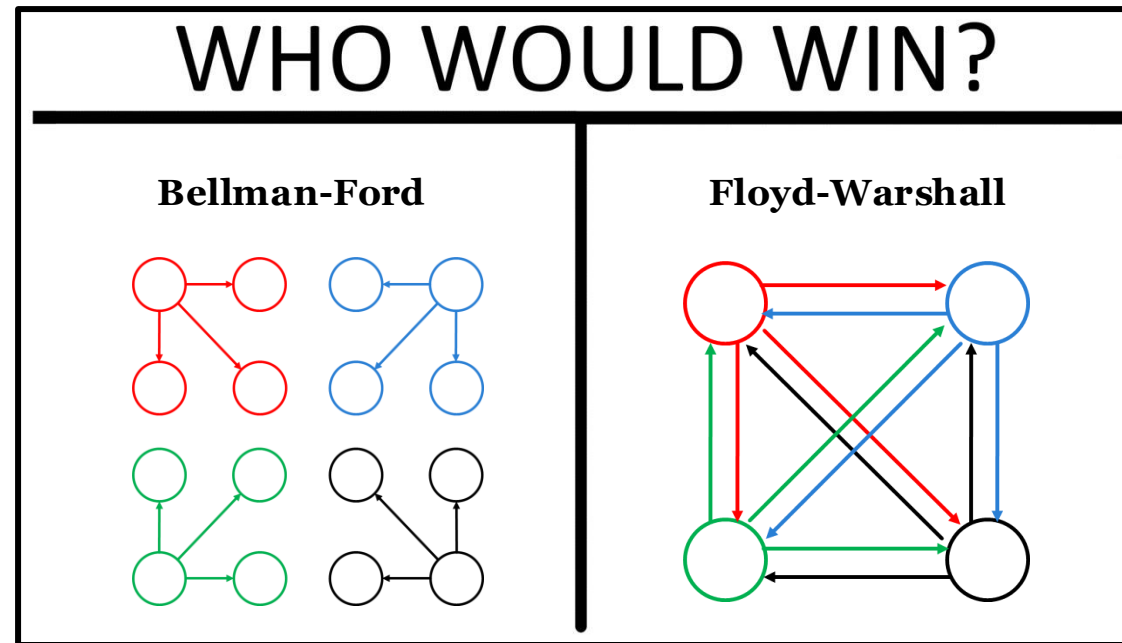


D4: Greedy Algorithms and Graph DP



Sec 17: Fr 11:30am-12:30pm FXB 1024
IA: Pearl Lin

Agenda

- ▶ Admin
- ▶ Graph DP problems and algorithms
 - ▶ All Pairs Shortest Path (Floyd-Warshall)
 - ▶ Single Source Shortest Path (Bellman-Ford)
- ▶ Greedy algorithms
- ▶ Worksheet

Admin

- ▶ HW 4 released, due Wednesday 9/30

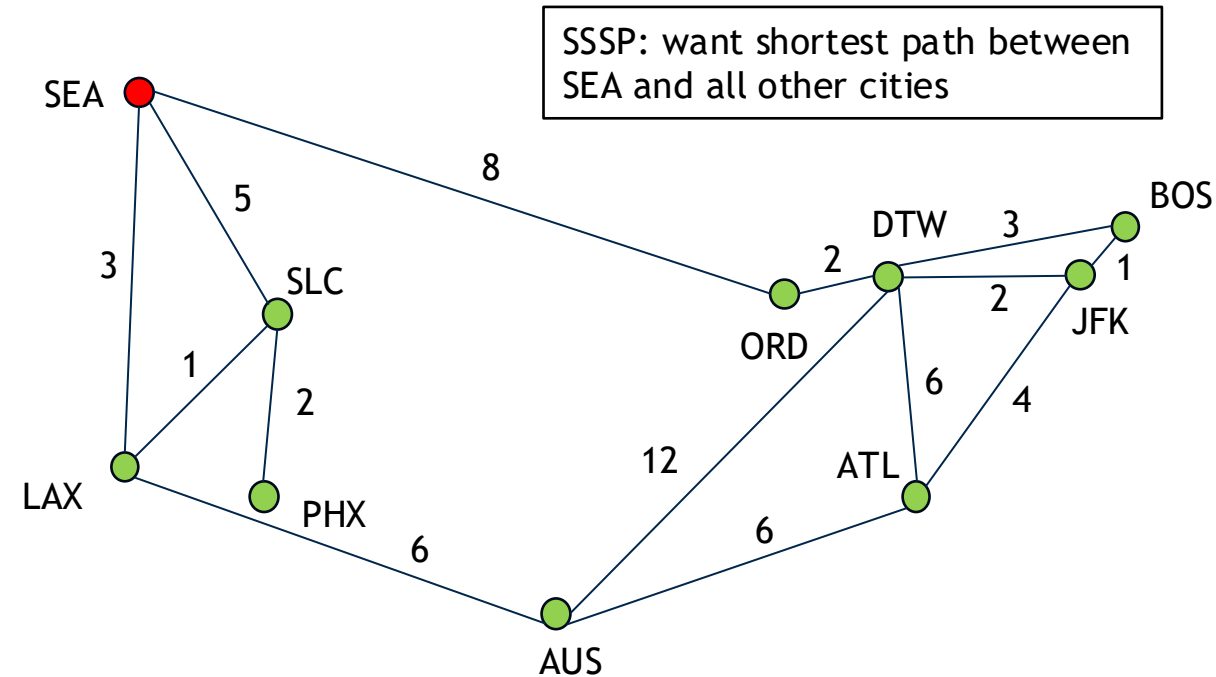
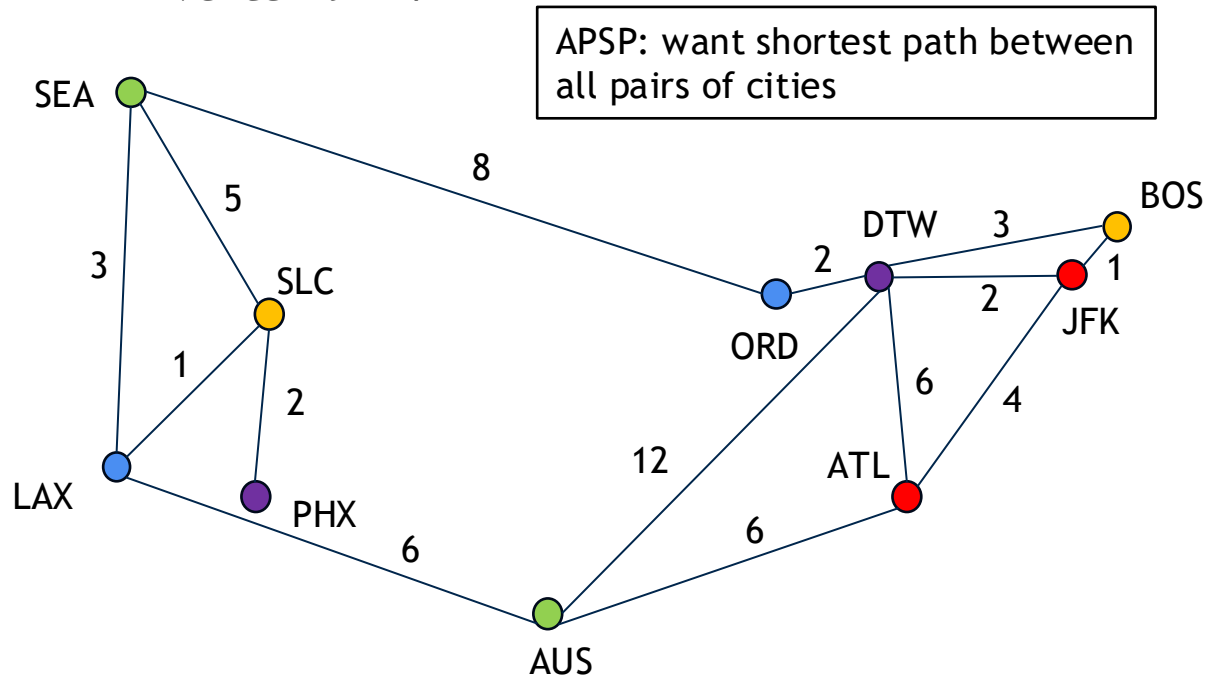
Graph DP

Shortest Path Problems

Note: APSP is a generalization of SSSP.

For weighted directed graphs $G = (V, E)$ (negative weights possible, but no negative cycles):

- ▶ **All Pairs Shortest Path (APSP)**: for every pair of vertices $s, t \in V$, find a shortest path from s to t
- ▶ **Single Source Shortest Path (SSSP)**: given a source vertex s , find a shortest path from s to every vertex $v \in V$



Floyd-Warshall (APSP)

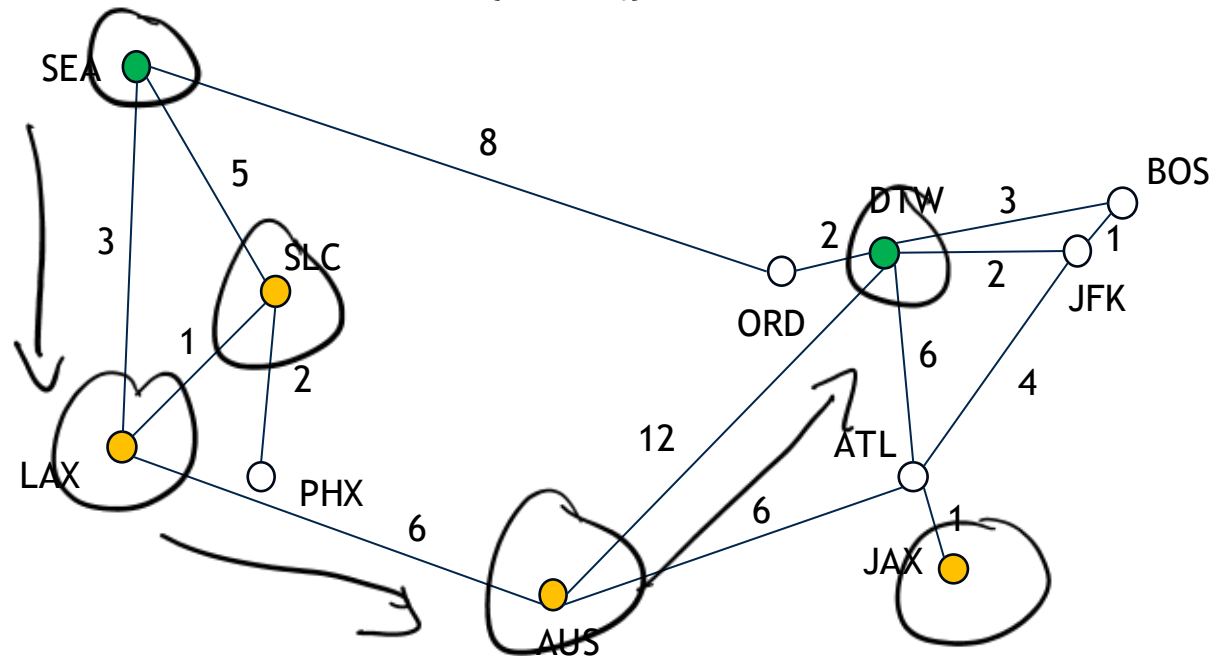
- ▶ DP Recipe #1:
 - ▶ Convert to value version
 - ▶ find the shortest path between $s, t \rightarrow$ find the **length** of the shortest path between s, t
 - ▶ Reduce the size of the problem
 - ▶ Introduce a size variable i : only consider paths that use intermediate vertices $\{v_1, \dots, v_i\}$

Define: Let $d^i(s, t)$ be the length of the shortest path between vertices s, t using only intermediate vertices $\{v_1, \dots, v_i\}$

Shortest path from SEA to DTW if we are only allowed to layover at:

- {SLC}
- {SLC, AUS}
- {SLC, AUS, LAX}
- {SLC, AUS, LAX, JAX}

$$\begin{aligned}d^1(s, t) &= \infty \\d^2(s, t) &= \infty \\d^3(s, t) &= 21 \\d^4(s, t) &= 21\end{aligned}$$



Floyd-Warshall (APSP) (continued)

$O(|V|^3)$

► DP Recipe #2:

► Consider just the vertex v_i . There are 2 possibilities:

► Case 1: v_i is part of the shortest path from $s \rightarrow t$. Let's **use it!**

- If we go from $s \rightarrow v_i$, then $v_i \rightarrow t$, we will have gone from $s \rightarrow t$
- Necessary condition: we must take the shortest path from $s \rightarrow v_i$ and the shortest path from $v_i \rightarrow t$. Otherwise, the path from $s \rightarrow t$ will not be a shortest path
- Necessary condition: The shortest paths from $s \rightarrow v_i$, $v_i \rightarrow t$ can only use intermediate vertices $\{v_1, \dots, v_{i-1}\}$
 - Why is it OK to not include v_i in the intermediary set anymore?
Having v_i as an intermediary vertex means that the paths $s \rightarrow v_i$, $v_i \rightarrow t$ could contain cycles. The only way for a path with a cycle to have shorter length than a path without one is if the cycle is negative (NOT ALLOWED!)

► So $d^i(s, t) = \underbrace{d^{i-1}(s, v_i)} + \underbrace{d^{i-1}(v_i, t)}$

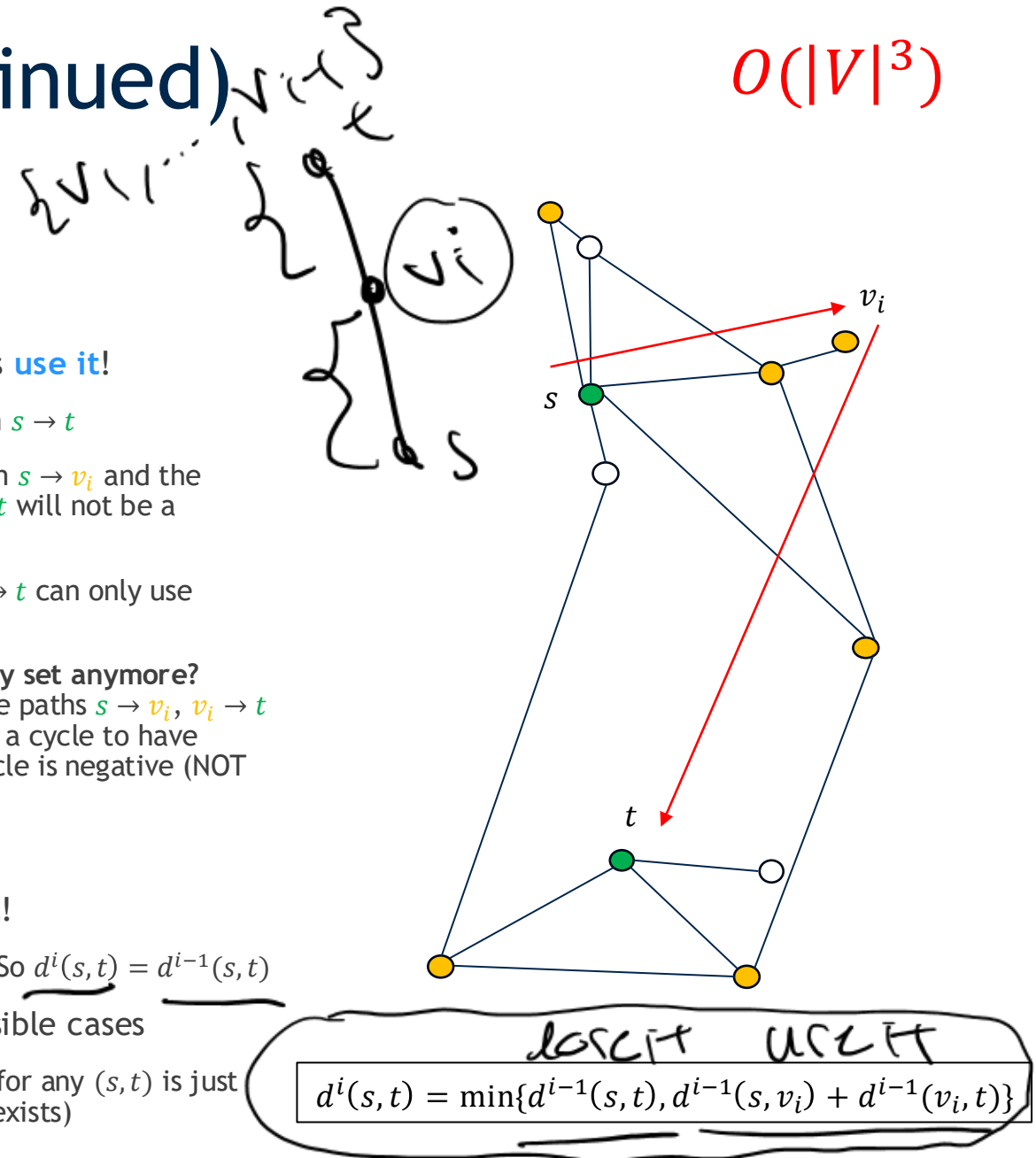
► Case 2: v_i is not part of the shortest path. Let's **lose it!**

- Same as only using intermediary vertices in $\{v_1, \dots, v_{i-1}\}$. So $\underline{d^i(s, t) = d^{i-1}(s, t)}$

► Putting it all together: we'll take the min of the 2 possible cases

- Base case when $i = 0$ (no intermediary vertices allowed) for any (s, t) is just the weight of edge between s, t (infinity if no such edge exists)

Define: Let $d^i(s, t)$ be the length of the shortest path between vertices s, t using only intermediate vertices from the set $\{v_1, \dots, v_i\}$



$$d^i(s, t) = \min\{d^{i-1}(s, t), d^{i-1}(s, v_i) + d^{i-1}(v_i, t)\}$$

Bellman-Ford (SSSP)

► DP Recipe #1:

► Convert to value version

► Find the shortest path between $s, v \rightarrow$ Find the **length** of the shortest path between s, v

► Reduce the size of the problem

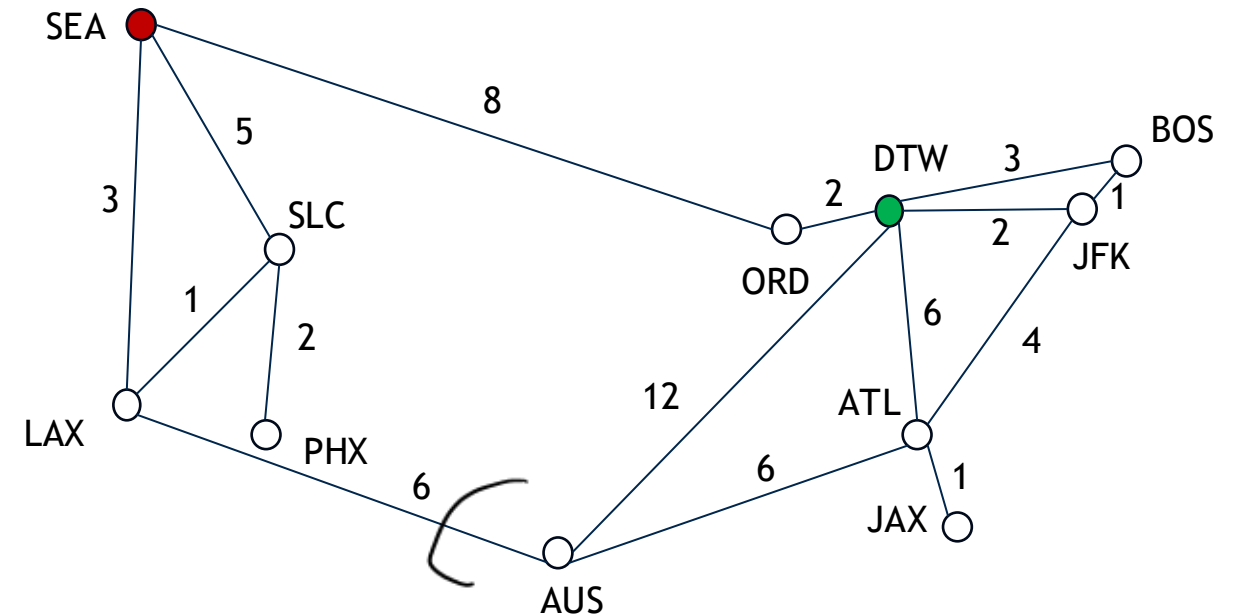
► Introduce a size variable i : only consider paths that use exactly i edges

s is fixed

Define: Let $d^i(s, v)$ be the **length of the shortest path between vertices s, v using exactly i edges**

Shortest path from SEA to DTW if we are only allowed to have:

- 0 layovers?
- 1 layover?
- 2 layovers?



Bellman-Ford (SSSP) (continued)

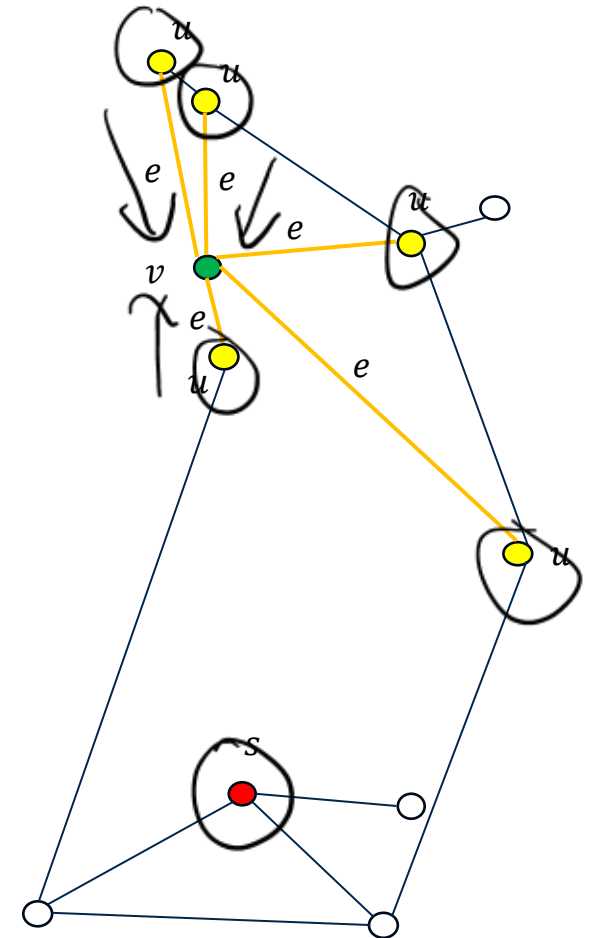
► DP Recipe #2:

► Consider just the **last edge** in a path from s to v

- Observation: this edge must start from some vertex u and end at vertex v . We'll call this edge e
- Shortest path from $s \rightarrow v$ = shortest path from $s \rightarrow u$ (using exactly $i - 1$ edges) + edge e
- Putting it all together: there are multiple candidates for edge e . We'll take the min out of all of them
 - $d^i(s, v) = \min_{(u,v) \in E} \{d^{i-1}(s, u) + \text{weight}(u, v)\}$
 - Base case when $i = 0$ is 0 if $s = v$, and ∞ if $s \neq v$

- To get the final answer, after all table filling is complete: iterate from $i = 0 \dots |E|$ and see which value of $d^i(s, v)$ is the smallest. The optimal value can always be found this way since a shortest path from s to v has to use one of $0, 1, 2, \dots, |E|$ edges

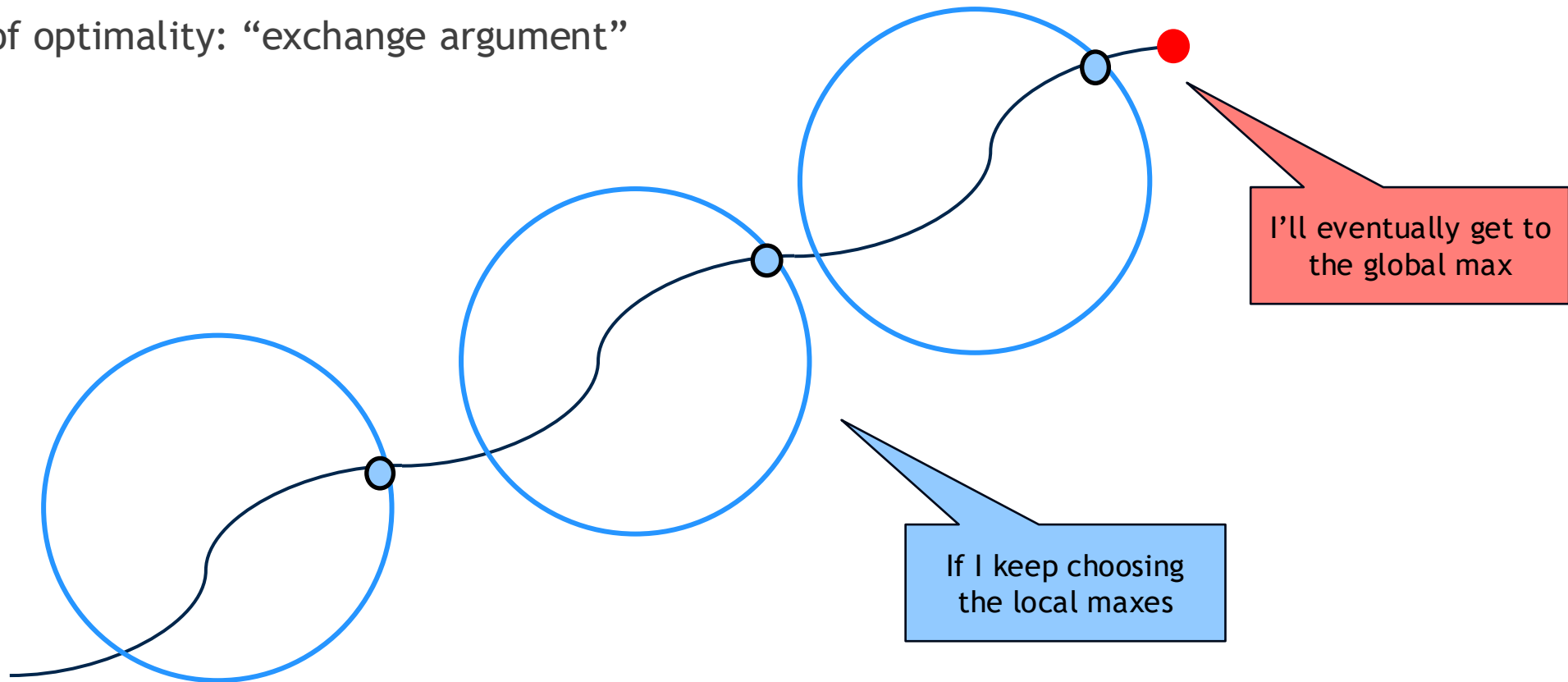
$$d^i(s, v) = \min_{(u,v) \in E} \{ \underbrace{d^{i-1}(s, u)} + \underbrace{\text{weight}(u, v)} \}$$



Greedy Algorithms

Overview

- ▶ Idea: Pick the **locally optimal** choice now, show that it leads you to the **globally optimal** solution
- ▶ Proof of optimality: “exchange argument”



Note: Sets are NOT ordered! Elements can be moved around and reindexed.

Exchange Argument

- ▶ Let **ALG** = $\{s_1, s_2, \dots, s_m\}$ be the solution outputted by the greedy algorithm
- ▶ Let **OPT** be some arbitrary optimal solution
- ▶ Idea: transform OPT into ALG by slowly incorporating more elements of ALG into OPT

Argue that a different optimal solution OPT' contains $S \cup \{s^*\}$.
Can be done by exchanging out an element $j^* \in U$ for s^*

$$\text{ALG} = \{s_1, s_2, \dots, s_{i-1}, s_i, s_{i+1}, \dots, s^*, \dots, s_m\}$$

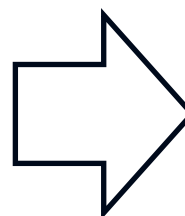
$$\text{OPT} = \{s_1, s_2, \dots, s_{i-1}, j_i, j_{i+1}, \dots, j^*, \dots, j_n\}$$

ALG and OPT share
these elements

ALG and OPT may differ
on these elements

$$\text{ALG} = S \cup T$$

$$\text{OPT} = S \cup U$$



$$\text{ALG} = \{s_1, s_2, \dots, s_{i-1}, s_i^*, s_{i+1}, \dots, s_m\}$$

$$\text{OPT}' = \{s_1, s_2, \dots, s_{i-1}, s_i^*, j_{i+1}, \dots, j_n\}$$

ALG and OPT share
these elements

ALG and OPT may differ
on these elements

$$\text{ALG} = S \cup \{s^*\} \cup T \setminus \{s^*\}$$

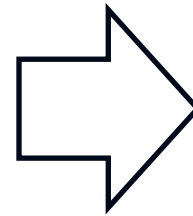
$$\text{OPT}' = S \cup \{s^*\} \cup U \setminus \{j^*\}$$

Exchange Argument (continued)

- ▶ Show that OPT' is actually an optimal solution by proving that
 - ▶ 1) It is a valid solution
 - ▶ 2) It is not a worse solution compared to OPT (so it is still optimal)
- ▶ Keeping repeating this process until ALG is fully contained within OPT (via induction)

$$\begin{aligned}\text{ALG} &= \{s_1, s_2, \dots, s_{i-1}, s_i^*, s_{i+1}, \dots, s_m\} \\ \text{OPT}' &= \{s_1, s_2, \dots, s_{i-1}, s_i^*, j_{i+1}, \dots, j_n\}\end{aligned}$$

After repeating
a bunch of
times...



$$\begin{aligned}\text{ALG} &= \{s_1, s_2, \dots, s_m\} \\ \text{OPT}' &= \{s_1, s_2, \dots, s_m, j_{m+1}, \dots\}\end{aligned}$$

$$\begin{aligned}\text{ALG} &= S \cup \{s^*\} \cup T \setminus \{s^*\} \\ \text{OPT}' &= S \cup \{s^*\} \cup U \setminus \{j^*\}\end{aligned}$$

$$\begin{aligned}\text{ALG} &= S \\ \text{OPT}' &= S \cup U'\end{aligned}$$

OPT' can technically have some stuff left over. But it already contains a full solution (ALG) so the rest can be "disregarded"

Worksheet

Q1

1. Longest Palindromic Subsequence

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

Given a string S , determine the length of a longest subsequence of S (not necessarily a substring) that is a palindrome. Recall that a palindrome is a string which is the same forwards and backwards (e.g., “racecar”).

Hint: try this recurrence definition

$LPS(i,j)$ = length of the longest palindromic subsequence using letters in $S[i,...,j]$

What can you say when $S[i] = S[j]$? $S[i] \neq S[j]$?



Q1 (annotations)

1. Longest Palindromic Subsequence

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

Given a string S , determine the length of a longest subsequence of S (not necessarily a substring) that is a palindrome. Recall that a palindrome is a string which is the same forwards and backwards (e.g., "racecar").

$$LPS(i, j) = \begin{cases} LPS(i+1, j-1) + 2, & S[i] = S[j] \\ \max(LPS(i+1, j), LPS(i, j-1)), & S[i] \neq S[j] \end{cases}$$

$$LPS(i, i) = 1 \quad \forall i \in [n].$$
$$LPS(i, j) = 0 \quad i > j.$$

Q1 (solution)

1. Longest Palindromic Subsequence

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

Given a string S , determine the length of a longest subsequence of S (not necessarily a substring) that is a palindrome. Recall that a palindrome is a string which is the same forwards and backwards (e.g., “racecar”).

Let $LPS(i, j)$ be a recurrence for the length of the longest palindromic subsequence using only letters in $S[i, \dots, j]$

There are 2 possible cases:

Case 1: $S[i] = S[j]$ (USE the 2 letters at the front and back)

Adds 2 to the length, plus the length of the longest palindromic subsequence using inner letters from $S[i + 1, \dots, j - 1]$

$$\Rightarrow LPS(i, j) = 2 + LPS(i + 1, j - 1)$$

Case 2: $S[i] \neq S[j]$ (LOSE one of the letters. Since they are not equal, we will never need them both)

We can lose either $S[i]$ or $S[j]$. We take the maximum out of these 2 choices.

$$\Rightarrow LPS(i, j) = \max(LPS(i + 1, j), LPS(i, j - 1))$$

Base cases:

When $i = j$, the palindrome is starting and ending at the same element, yielding a 1-character long string. It's still a palindrome!

$$\Rightarrow LPS(i, i) = 1$$

When $j < i$, the starting index is past the ending index, producing an empty string.

$$\Rightarrow LPS(i, j) = 0$$

$$LPS(i, j) = \begin{cases} 2 + LPS(i + 1, j - 1), & S[i] = S[j] \\ \max(LPS(i + 1, j), LPS(i, j - 1)), & S[i] \neq S[j] \\ LPS(i, i) = 1, LPS(i, j) = 0, & i > j \end{cases}$$

Q3

$d[s, v]$ = ~~the~~ weight of lowest
cost path $s \rightarrow v$ using
exactly i
edges.

3. Vertex-weighted single-source lowest-cost paths (VWSSLCP).

In this problem we consider a variation of shortest-paths DP. Suppose we are given a graph $G = (V, E)$ and a cost function $c: V \rightarrow \mathbb{R}$ that maps each *vertex* to a cost. The cost of a path is simply the total cost of all the vertices on the path (including the starting and ending vertex). Again, we assume that there is **no negative-cost cycle** (although individual vertex costs can still be negative).

Given a source vertex $s \in V$, our goal is to find the *minimum path cost* from s to each vertex $v \in V$. Give a recurrence relation for this problem. Remember to include appropriate base cases(s).

SSSP

Hint: what lecture problem is this similar to?
Hint 2: it's also in these slides

Q3 (annotations)

$d^{(i)}(s, v)$ = weight of lowest
cost path $s \rightarrow v$ using exactly i edges

$$d^{(i)}(s, v) = \min_{(u, v) \in E} \left(d^{(i-1)}(s, u) + c(v) \right)$$

$\approx$

weight of
($i-1$) edged
min cost path
from $s \rightarrow u$

weight of
(i) edged
min cost path
from $s \rightarrow v$

3. Vertex-weighted single-source lowest-cost paths (VWSSLCP).

In this problem we consider a variation of shortest-paths DP. Suppose we are given a graph $G = (V, E)$ and a cost function $c: V \rightarrow \mathbb{R}$ that maps each *vertex* to a cost. The cost of a path is simply the total cost of all the vertices on the path (including the starting and ending vertex). Again, we assume that there is **no negative-cost cycle** (although individual vertex costs can still be negative).

Given a source vertex $s \in V$, our goal is to find the *minimum path cost* from s to each vertex $v \in V$. Give a recurrence relation for this problem. Remember to include appropriate base cases(s).

Q3 (solution)

Solution: We notice that the key observations from [Section 1.2](#) for shortest paths *still apply* to lowest-cost paths. We can use the same insights behind the Bellman-Ford recurrence in the lowest-cost path recurrence.

We give our recurrence by modifying Bellman-Ford. Similarly, we define $d^{(i)}(s, v)$ to be the cost of a lowest-cost path from s to v that uses **exactly** i edges, or ∞ if there is no such

path. Then

$$d^{(0)}(s, v) = \begin{cases} c(s) & \text{if } s = v \\ \infty & \text{if } s \neq v, \end{cases}$$
$$d^{(i)}(s, v) = \min_{(u,v) \in E} \left\{ d^{(i-1)}(s, u) + c(v) \right\}.$$

For the base case (i.e., lowest-cost paths with 0 edges), if $s = v$, a path that starts and ends at the same node s has cost $c(s)$. Otherwise, we cannot go from s to v using 0 edges, so the cost is ∞ .

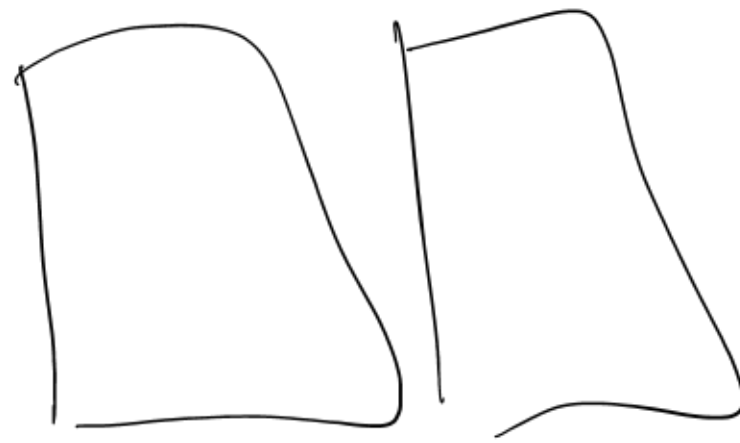
In the recurrence (lowest-cost paths with exactly i edges), we know that to arrive at v , we must have come from some immediate predecessor vertex u , where $(u, v) \in E$. And by definition, the lowest cost to go from s to u using $i - 1$ edges is $d^{(i-1)}(s, u)$. So, the lowest-cost s -to- v path that uses exactly i edges is given by the minimum over all these options, which is captured in the above recurrence.

3. Vertex-weighted single-source lowest-cost paths (VWSSLCP).

In this problem we consider a variation of shortest-paths DP. Suppose we are given a graph $G = (V, E)$ and a cost function $c: V \rightarrow \mathbb{R}$ that maps each *vertex* to a cost. The cost of a path is simply the total cost of all the vertices on the path (including the starting and ending vertex). Again, we assume that there is **no negative-cost cycle** (although individual vertex costs can still be negative).

Given a source vertex $s \in V$, our goal is to find the *minimum path cost* from s to each vertex $v \in V$. Give a recurrence relation for this problem. Remember to include appropriate base cases(s).

Q4A



4. Copying homework problems.

You are copying n problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.

- (a) Describe a greedy algorithm to minimize the number of pages you need.

Q4A (annotated)

4. Copying homework problems.

You are copying n problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.

- (a) Describe a greedy algorithm to minimize the number of pages you need.

Q4A (solution)

4. Copying homework problems.

You are copying n problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.

- (a) Describe a greedy algorithm to minimize the number of pages you need.

Copy as many problems as you can onto a page before moving on to the next one

Q4B

4. Copying homework problems.

You are copying n problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.

- (b) Prove that the proposed greedy algorithm is correct. In other words, prove that the proposed algorithm uses the minimum number of pages, while satisfying the constraints given in the problem.

Hint: set up exchange argument as follows:

Let $ALG = (p_1, p_2, \dots, p_{diff-1}, p_{diff}, p_{diff+1}, \dots, p_m)$, the non-decreasing sequence of page numbers the algorithm assigns each problem

Let $OPT = (p_1, p_2, \dots, p_{diff-1}, k_{diff}, k_{diff+1}, \dots, k_n)$, the optimal non-decreasing sequence of page numbers assigned to each problem

What element should you exchange into OPT ? Which element in OPT will get replaced?

Q4B (annotations)

4. Copying homework problems.

You are copying n problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.

- (b) Prove that the proposed greedy algorithm is correct. In other words, prove that the proposed algorithm uses the minimum number of pages, while satisfying the constraints given in the problem.

Q4B (solution)

4. Copying homework problems.

You are copying n problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.

- (b) Prove that the proposed greedy algorithm is correct. In other words, prove that the proposed algorithm uses the minimum number of pages, while satisfying the constraints given in the problem.

Sketch of proof:

Let $ALG = (p_1, p_2, \dots, p_{diff-1}, p_{diff}, p_{diff+1}, \dots, p_m)$, the non-decreasing sequence of page numbers the algorithm assigns each problem

Let $OPT = (p_1, p_2, \dots, p_{diff-1}, k_{diff}, k_{diff+1}, \dots, k_n)$, the optimal non-decreasing sequence of page numbers assigned to each problem

Replace k_{diff} with p_{diff} , so $OPT' = \{p_1, \dots, p_{diff-1}, p_{diff}, k_{diff+1}, \dots, k_n\}$

OPT' is a valid solution since $p_{diff} \leq k_{diff} \leq \{k_{diff+1}, \dots, k_n\}$, given that OPT is a valid solution and p_{diff} is the earliest possible page that the problem could have fit on. By our algorithm, it is $\geq \{p_1, \dots, p_{diff-1}\}$. So indeed, the sequence of page numbers is non-decreasing.

OPT' is not a worse solution since it does not increase the page number of the final problem n

Additional Slides

Floyd-Warshall Pseudocode/Runtime

```
ALLPAIRSSHORTESTPATH ( $G(V,E)$ ):
```

```
    allocate table[1... $|V|$ ][1... $|V|$ ][0... $|V|$ ]
```

```
    for  $s = 1$  to  $|V|$  do:
```

```
        for  $t = 1$  to  $|V|$  do:
```

```
            if  $s = t$ :
```

```
                table[s][t][0] = 0
```

```
            else if  $(s,t) \in E$  :
```

```
                table[s][t][0] = weight(s,t)
```

```
            else:
```

```
                table[s][t][0] =  $\infty$ 
```

```
    for  $s = 1$  to  $|V|$  do:
```

```
        for  $t = 1$  to  $|V|$  do:
```

```
            for  $i = 1$  to  $|V|$  do:
```

```
                table[s][t][i] = min{table[s][t][i-1],  
                                     table[s][i][i-1] + table[i][t][i-1]}
```

```
    return table[s][t][ $|V|$ ] for all  $(s,t)$ 
```

$O(|V|^2)$ cells * $O(1)$ time to fill each cell
= $O(|V|^2)$

+ = $O(|V|^3)$

$O(|V|^3)$ cells * $O(1)$ time to fill each cell
= $O(|V|^3)$

Take
dominating
term

Bellman-Ford Pseudocode/Runtime

SINGLESOURCESHORTESTPATH ($G(V,E),s$):

allocate table[1... $|V|$][0... $|V|-1$]

for $v = 1$ to $|V|$ do:

if $s \neq v$ do:

table[v][0] = ∞

else:

table[v][0] = 0

for $v = 1$ to $|V|$ do:

for $i = 0$ to $|V| - 1$ do:

table[v][i] = min_ $\{(u,v) \in E$

(table[u][i-1] + w(u,v))

return max_ $\{0 \leq i \leq |V| - 1\}$ (table[v][i]) for all v

$O(|V|)$ cells * $O(1)$ time to fill each cell
= $O(|V|)$

+ = $O(|V||E|)$

$O(|V|^2)$ cells * $O(\frac{|E|}{|V|})$ time to fill each cell =
 $O(|V||E|)$

Why $O(\frac{|E|}{|V|})$? We are iterating through all edges that go from some vertex u to v , this can be at maximum $|E|$ edges. We are splitting this work up across $|V|$ loop iterations, hence why we divide by $|V|$.

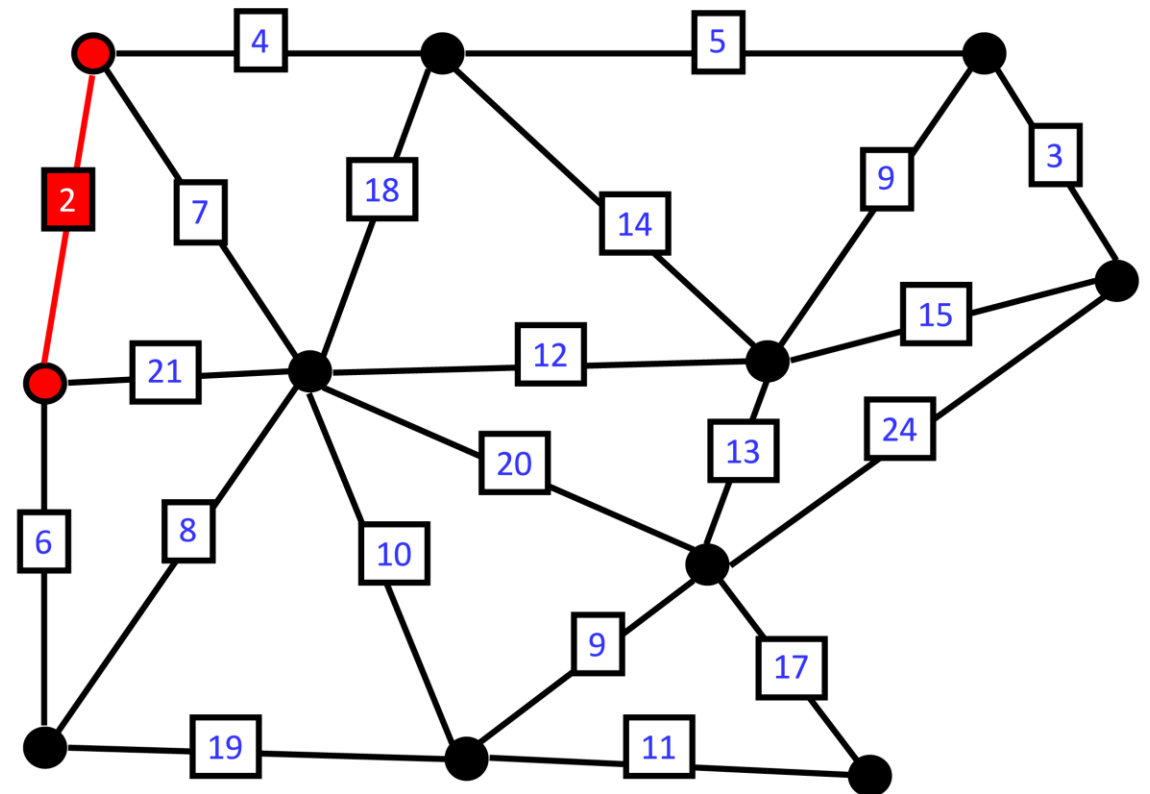
+

Linear pass over $O(|V|)$ cells = $O(|V|)$

Take dominating term

Example: Minimum Spanning Tree

- ▶ Problem: Given a weighted, connected, undirected graph, find the **minimum spanning tree (MST)** that reaches every vertex and has minimal cost
- ▶ Greedy algorithm idea (**Kruskal's algorithm**):
 - ▶ Sort edges by weight
 - ▶ Examining edges in order, greedily select any edge that **does not create a cycle** when added to the previously selected edges

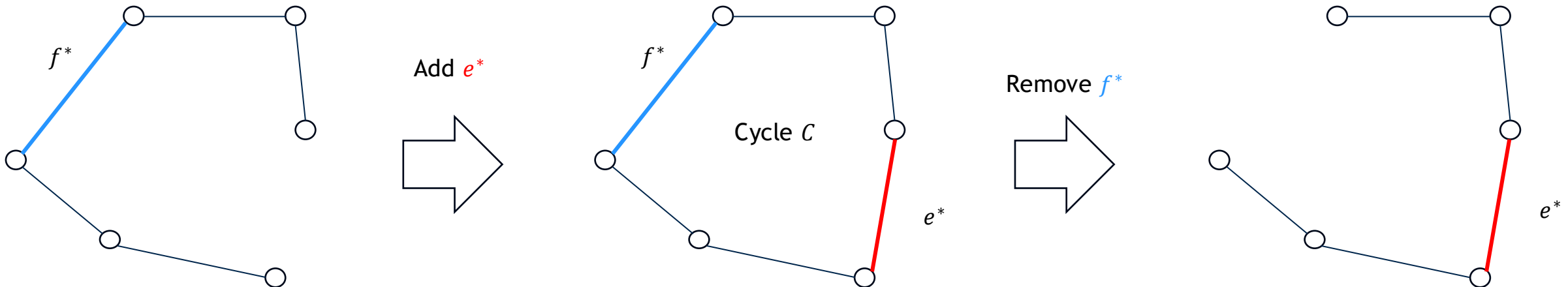


Example: Minimum Spanning Tree (continued)

- ▶ Use the exchange argument to prove that Kruskal indeed outputs an MST
- ▶ Let $\mathbf{ALG} = \{e_1, e_2, \dots, e_{i-1}, e_i^*, e_{i+1}, \dots, e_m\}$ be the selection of edges from Kruskal
- ▶ Let $\mathbf{OPT} = \{e_1, e_2, \dots, e_{i-1}, f_i, f_{i+1}, \dots, f^*, \dots, f_n\}$ be the selection of edges that creates an (arbitrary) MST
- ▶ Idea: Exchange $f^* \in \{f_i, f_{i+1}, \dots, f_n\}$ for e_i^* :
 - ▶ $f^* = ???$

Example: Minimum Spanning Tree (continued)

- ▶ Observation: since OPT is a tree, adding e^* to OPT creates a cycle (call it $\mathcal{C}$)
- ▶ $\{e_1, e_2, \dots, e_{i-1}\} \cup \{e^*\}$ cannot contain a cycle, given that these edges were selected by Kruskal's and the algorithm only selects edge weights if they don't create a cycle.
- ▶ So, there must exist an element some element $f^* \in \mathcal{C}$ such that $f^* \notin \{e_1, e_2, \dots, e_{i-1}\} \cup \{e^*\}$. It follows that $f^* \in \{f_i, f_{i+1}, \dots, f_n\}$
- ▶ Idea: add e^* to OPT, then remove f^* to fix the cycle $\mathcal{C}$



Example: Minimum Spanning Tree (continued)

- ▶ Exchange argument #1: OPT' is a valid spanning tree
 - ▶ After removing an element from a cycle, there is still a way to get to all vertices in the cycle ($\checkmark$ spanning)
 - ▶ Removing an element from a cycle fixes the cycle ($\checkmark$ tree)
- ▶ Exchange argument #2: OPT' is not a worse solution, i.e., $\text{totalCost}(\text{OPT}') \leq \text{totalCost}(\text{OPT})$
 - ▶ Observation: f^* does not create a cycle with $\{e_1, e_2, \dots, e_{i-1}\}$ since all these elements are in OPT , and OPT is a valid MST
 - ▶ So f^* was available to select when Kruskal was selecting e^* . But Kruskal chose e^* over f^* !
 - ▶ This must mean that $\text{cost}(e^*) \leq \text{cost}(f^*)$. Since OPT' is the same as OPT save for replacing f^* with e^* , we necessarily must have $\text{totalCost}(\text{OPT}') \leq \text{totalCost}(\text{OPT})$

$$\text{OPT} = \{e_1, \dots, e_{i-1}, f_i, f_{i+1}, \dots, f^*, \dots, f_n\} \quad \Rightarrow \quad \text{OPT}' = \{e_1, \dots, e_{i-1}, e_i^*, f_{i+1}, \dots, f_n\}$$